Sunbeam Institute of Information Technology
Pune and Karad

Embedded and RTOS (FreeRTOS) Training At RIT College, Islampur

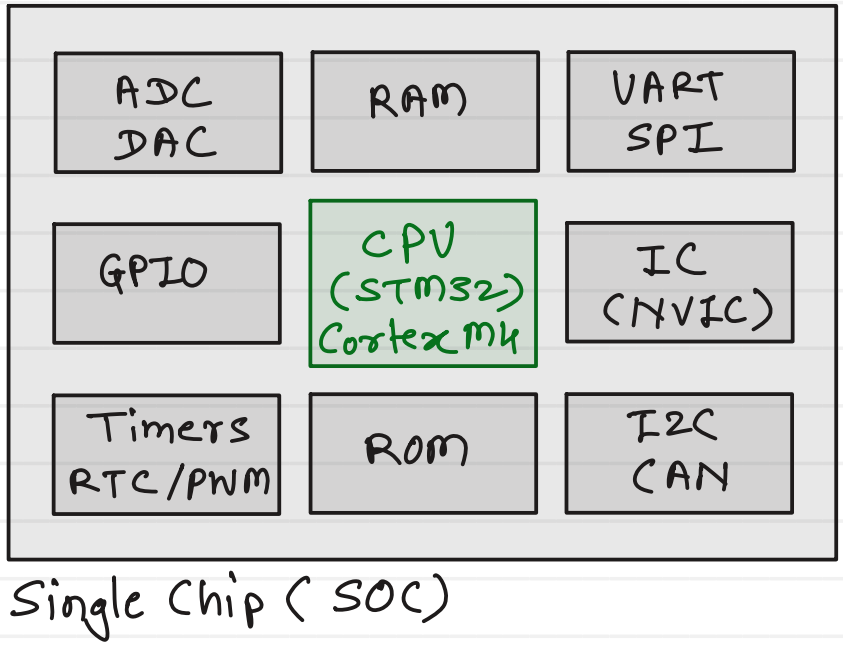
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Microcontroller (MCU)

- A microcontroller is a programmable electronic device designed to control specific tasks in embedded systems.

- A **Microcontroller (MCU)** is a small integrated circuit that contains all on a **single chip**:
  - **CPU (Processor)**
  - **Memory (RAM, ROM/Flash)**
  - **Input/Output (GPIO) Ports**
  - **Peripherals (Timers, UART, SPI, I2C, ADC, etc.)**



- **Main Components**
  - **CPU**
    - Executes instructions
    - Performs arithmetic and logical operations
  - **Flash / ROM**
    - Stores program code
    - Non-volatile memory
  - **RAM**
    - Stores variables and temporary data
    - Volatile memory
  - **Peripherals**
    - GPIO, Timers/Counters, UART, SPI, I2C, ADC, PWM, Watchdog Timer

Embedded C

- **Embedded C** is an extension of the C programming language used for developing software for **embedded systems** such as microcontrollers, microprocessors, and IoT devices.
- It includes standard C features along with hardware-specific programming techniques.

- **Why Not to Use Normal C?**
  - Standard C is designed for general-purpose computers.
  - Embedded systems require:
    - Direct hardware access
    - Register manipulation
    - Interrupt handling
    - Memory-mapped I/O
    - Real-time operation
  - Embedded C provides these capabilities.

- **Features of Embedded C**
  - **Hardware register access and memory mapping**
    - Pointers to hardware
    - Bitwise operations
    - Structure mapping
  - **Embedded Storage Modifiers**
    - volatile, const, static
  - **Interrupts and Asynchronous Execution**
    - Interrupt Service Routines
    - Critical sections
    - Atomicity
  - **Memory Layouts and Custom Allocation**
    - Linker Symbols
    - Complier Attributes

Bitwise Operators

- Bitwise operators perform operations on the **individual bits** of an integer.
- **Why Use Bitwise Operators?**
  - Efficient hardware control
  - Register manipulation
  - Device driver development
  - Embedded systems programming
  - Memory optimization
- List of bitwise operators

Operator	Name	Example
&	Bitwise AND	a & b
|	Bitwise OR	a | b
^	Bitwise XOR	a ^ b
~	Bitwise NOT	~a
<<	Left Shift	a << n
>>	Right Shift	a >> n

```
  10 - 1010
& 12 - 1100
-----------
        1000
```

```
  10 - 1010
| 12 - 1100
-----------
        1110
```

```
  10 - 1010
^ 12 - 1100
-----------
        0110
```

```
~ 10 - 1010
-----------
        0101
```

```
~ 12 - 1100
-----------
        0011
```

integer datatypes : char, short, int, long
type modifiers : signed or unsigned

Left shift (<<)

unsigned char var = 0xA4

```
0xA4 :  1010 0100
<< 2 :
-------------------
        1001 0000
```

signed char var = 0xA4

```
0xA4 :  1010 0100
<< 2 :
-------------------
        1001 0000
```

Right shift (>>)

unsigned char var = 0xA4

```
0xA4 :  1010 0100
<< 2 :
-------------------
        0010 1001
```

signed char var = 0xA4

```
0xA4 :  1010 0100
<< 2 :
-------------------
        1110 1001
```

check n^{th} bit of register

```
regr = 0xA4
                7 6 5 4   3 2 1 0
0xA4  :         1 0 1 0   0 1 0 0
  &             0 0 1 0   0 0 0 0
-------------------------------------
                0 0 1 0   0 0 0 0   (non zero)

                7 6 5 4   3 2 1 0
0xA4  :         1 0 1 0   0 1 0 0
  &             0 1 0 0   0 0 0 0
-------------------------------------
                0 0 0 0   0 0 0 0   (zero)
```

```
if ( regr & (1<<n) )
    // nth bit is 1 (set)
else
    // nth bit is 0 (clear)
```

```
0xA4  :  1 0 1 0   0 1 0 0
mask  :  1 0 0 0   0 0 0 0   - non zero = 1
         0 1 0 0   0 0 0 0   - zero     = 0
         0 0 1 0   0 0 0 0   - non zero = 1
                 :
         0 0 0 0   0 0 0 1   - zero     = 0
```

```
mask = 0x80;
while(mask) {
    if( value & mask)
        printf("1");
    else
        printf("0");
    mask = mask >> 1;
}
```

set n^{th} bit of a register

```
regr = 0xA4
0xA4 : 1010 0100
     | 0100 0000
-------------------
       1110 0100
```

```
regr = 0xA4
0xA4 : 1010 0100
     | 0010 0000
-------------------
       1010 0100
```

```
regr = regr | (1 << n)
```
or
```
regr |= (1 << n)
```

clear n^{th} bit of a register

```
regr = 0xA4
0xA4 : 1010 0100
     ~ 0100 0000
     & 1011 1111
-------------------
       1010 0100
```

```
regr = 0xA4
0xA4 : 1010 0100
     ~ 0010 0000
     & 1101 1111
-------------------
       1000 0100
```

```
regr = regr & ~(1 << n)
```
or
```
regr &= ~(1 << n)
```

toggle n^{th} bit of a register

```
regr = 0xA4
0xA4 : 1010 0100
     ^ 0100 0000
-------------------
       1110 0100
```

```
regr = 0xA4
0xA4 : 1010 0100
     ^ 0010 0000
-------------------
       1000 0100
```

```
regr = regr | (1 << n)
```
or
```
regr |= (1 << n)
```

#define BV(n) (1<<(n))

set bits from 4 to 7

```
regr = 0xA4
0xA4 : 1010 0100
       1000 0000  (1<<7)
       0100 0000  (1<<6)
       0010 0000  (1<<5)
     | 0001 0000  (1<<4)
-------------------------
mask: 1111 0000
     |
-------------------------
       1111 0100
```

```
regr = regr | BV(7) | BV(6) | BV(5) | BV(4);
```
or
```
regr |= BV(7) | BV(6) | BV(5) | BV(4);
```

clear bits from 4 to 7

```
regr = 0xA4
0xA4 : 1010 0100
       1000 0000  (1<<7)
       0100 0000  (1<<6)
       0010 0000  (1<<5)
     | 0001 0000  (1<<4)
-------------------------
~      1111 0000
-------------------------
mask: 0000 1111
     &
-------------------------
       0000 0100
```

```
regr = regr & ~(BV(7) | BV(6) | BV(5) | BV(4));
```
or
```
regr &= ~(BV(7) | BV(6) | BV(5) | BV(4));
```

Function Pointer

- A **function pointer** is a pointer that stores the **address of a function**.
- Just as a normal pointer stores the address of a variable, a function pointer stores the address of a function.

- Where Function Pointers are used?
  - Callback functions
  - Menu-driven programs
  - State machines
  - Interrupt service routines
  - Device drivers
  - Generic libraries (qsort(), signal handlers, etc.)

function declaration :

```
<return type> funName([List of datatypes of args]);
e.g. int addition(int, int);
     int factorial(int);
```

function pointer declaration :

```
<return type> (*ptrName)([List of datatypes of args]);
e.g. int (*paddition)(int, int);
     int (*pfactorial)(int);
```

Vectored Interrupts

- action to be taken on a interrupt is always written into ISR (function)
- addresses of all the ISR's are fix and stored into a table called as IVT (Interrupt Vector Table)
- IVT is always placed in very first address of the RAM (memory)

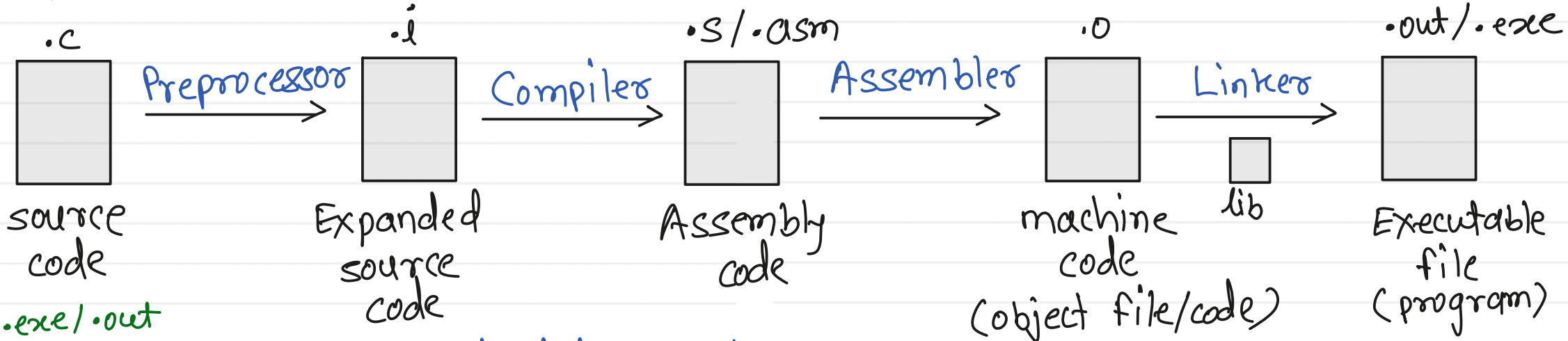
Non vectored interrupts

- address of ISRs are not fixed and also not collected into IVT
- address will be searched when interrupt will arrive, so interrupt delivery will be slower

C program and its compilation

→ GCC - GNU C compiler

- A **toolchain** is a collection of software tools used to convert source code into an executable program.
- Typical tools are preprocessor, compiler, assembler, linker and debugger.
- Program – collection / set of instructions to the Machine (CPU)



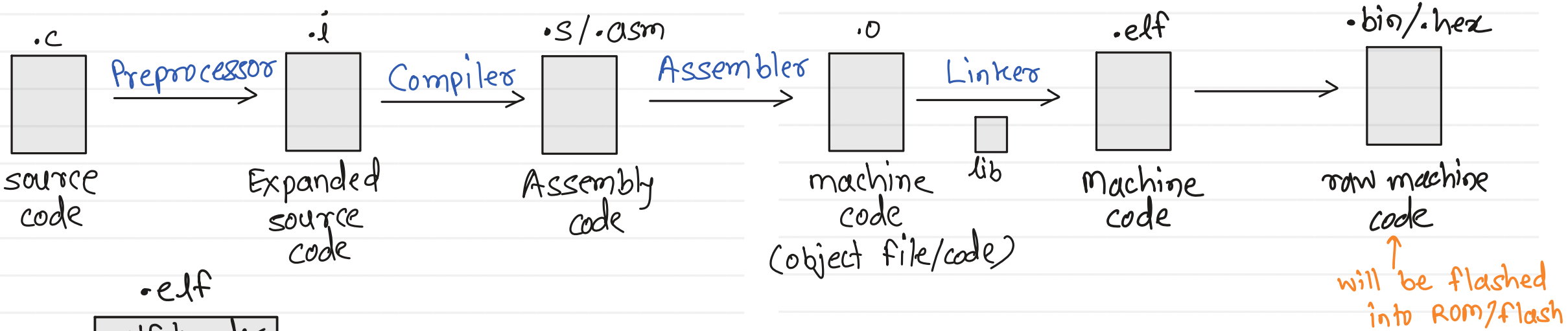
.exe / .out

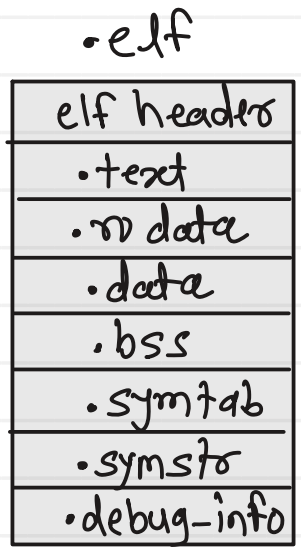
Exe header
.text
.ro data
.data
.bss
.symtab
.symstr
.debug_info

→ info required to execute a program
→ instructions / functions in machine code format
→ read only data (string constants)
→ initialized static & global variables
→ uninitialized static & global variables
→ symbol table - info about variables and functions
→ symbol strings
→ info about debugging

Embedded C program and its compilation

- In embedded systems, **a toolchain** is used to transform **Embedded C source code** into machine code that runs on a microcontroller.
- Typical tools are preprocessor, compiler, assembler, linker, debugger and objcopy etc





Toolchain : arm-none-eabi-gcc

- arm : arch
- none : no vendor or OS
- eabi : Embedded ABI
- gcc : GNU C Compiler